DOCUMENT 2 / 3 — ARCHITECTURE

Architecture technique

Stack, Clean Architecture, authentification, pipeline de récupération des prix et assistant IA — état réel de l'implémentation (vérifié dans le repo), avec les écarts connus par rapport à la documentation Obsidian d'origine.

.NET 8/9 · Clean Architecture | Angular 19 | PostgreSQL 16 · Redis 7

Client
Gérant boutique Pokémon (30 m²)

Périmètre
Stack, architecture, auth, data, IA

Source
Coffre Obsidian + repo `Pokekool`

Généré le
31 août 2026

SOMMAIRE

Contenu du document

01 **Stack technique** — Vue d'ensemble des couches, technologies réellement en place

02 **Clean Architecture** — Les 4 couches du backend et la règle de dépendance

03 **Authentification** — JWT + Refresh Token, rotation, sécurité

04 **Sources de données & scraping** — Providers de prix, pipeline, règle « pas d'IA dans le scraping »

05 **Assistant IA — AI-Brain** — Base de connaissance RAG pour le scan de cartes

06 **Environnement dev & points d'attention** — Ports locaux, dette technique héritée du template

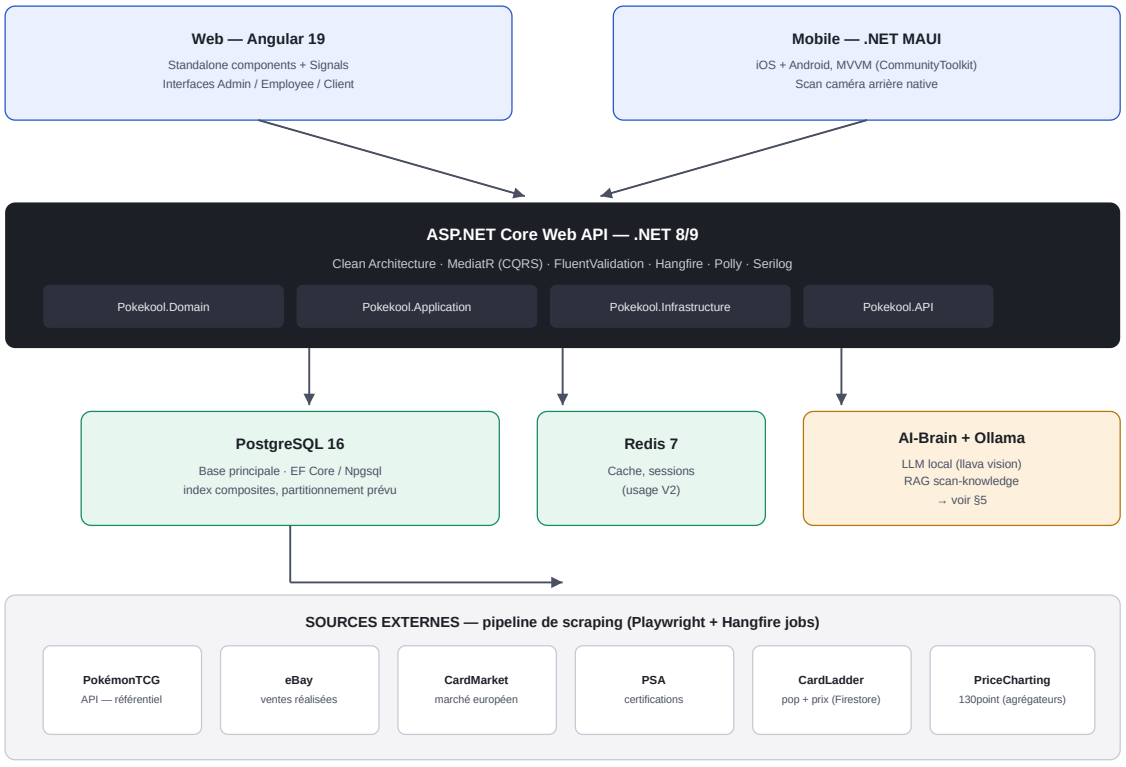
01 — STACK

Stack technique

Architecture 3-tiers classique — client web Angular et client mobile .NET MAUI consommant une API .NET en Clean Architecture, adossée à PostgreSQL et Redis. Un pipeline de scraping alimente les prix depuis des sources externes, et un module IA locale (Ollama) assiste le scan de collection.

ÉCART DOCUMENTATION ↔ IMPLÉMENTATION RÉELLE

La vision d'origine (`01-Vision/Stack-Technique.md` et `ARCHITECTURE.md` « document fondateur ») ciblait un frontend **React + TypeScript**. L'implémentation réelle, confirmée dans le repo (`README.md`, `src/web/poke-web`, `MODULE-CLIENT.md`), utilise **Angular 19 (standalone + Signals)**. Le pivot n'est documenté nulle part explicitement dans le vault — à noter et à corriger dans `Stack-Technique.md` pour éviter toute confusion à la reprise.



Architecture système — client Angular/MAUI, API .NET en Clean Architecture, stockage et sources externes.

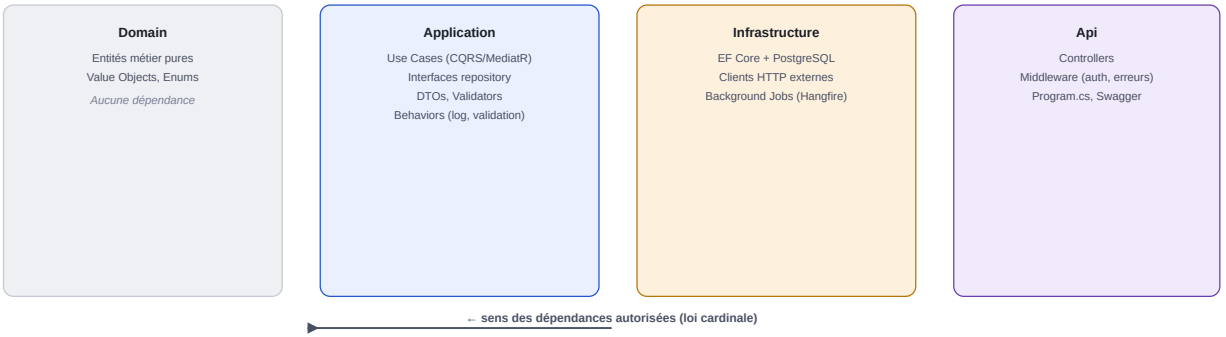
Technologies par couche

Couche	Technologies	Rôle
Backend	.NET 8/9 · ASP.NET Core · EF Core · MediatR · FluentValidation · Hangfire · Polly · Serilog · Swagger	API REST, CQRS, jobs planifiés, retry sur appels externes, logs structurés
Web	Angular 19 (standalone + Signals) · TypeScript	Interfaces Admin, Employee, Client
Mobile	.NET MAUI (iOS/Android) · CommunityToolkit.Mvvm · `Pokekool.Shared`	Scan caméra native, consultation rapide en boutique
Base de données	PostgreSQL 16 · Npgsql	Stockage principal — voir doc « Schéma BDD »
Cache	Redis 7	Prévu pour cache prix / sessions (usage V2)
IA locale	Ollama (llava vision) + base de connaissance RAG `ai-brain/`	Scan photo, reconnaissance de cartes
Scraping	Playwright (headless), XHR intercept, CSS eval	Récupération des prix / population multi-sources
Design system	`tokens.json` unique → export CSS (Angular) + XAML (MAUI)	Cohérence visuelle web/mobile
Infra (prévu)	Docker Compose (dev) · GitHub Actions · Railway/Render	CI/CD et hébergement — à finaliser

02 — BACKEND

Clean Architecture

Décision actée (**ADR-001**, juin 2026) : Clean Architecture à 4 couches, avec une règle de dépendance stricte — chaque couche ne connaît que celle immédiatement en dessous d'elle.



4 couches, dépendance à sens unique vers le Domain — permet de changer d'infrastructure sans toucher au métier.

POURQUOI CE CHOIX (ADR-001)

Testabilité du Domain/Application sans infrastructure, changement de base de données possible sans toucher au métier, séparation claire des responsabilités, onboarding facilité.

COMPROMIS ASSUMÉ

Plus de fichiers et de boilerplate au démarrage ; courbe d'apprentissage si l'équipe ne connaît pas le pattern. Alternative « Minimal API flat » écartée (dette technique rapide à l'échelle).

03 — SÉCURITÉ

Authentification — JWT + Refresh Token

Décision actée (**ADR-002**, juin 2026) : API stateless, JWT court en mémoire côté client + Refresh Token longue durée en cookie `HttpOnly`, avec rotation à chaque renouvellement.

Token	Stockage	Durée
JWT Access Token	Mémoire (variable JS) — jamais en localStorage	15 minutes
Refresh Token	Cookie `HttpOnly Secure SameSite=Strict`	7 jours, rotation à chaque usage



Si l'ancien Refresh Token révoqué est réutilisé → vol de token détecté (signal de compromission).

Mot de passe hashé BCrypt (cost 12) · JWT signé HS256 · logout = révocation serveur.

Endpoints

Méthode	Endpoint	Description
`POST`	`/auth/register`	Création de compte (rôle `client` par défaut)
`POST`	`/auth/login`	Connexion, retourne le JWT
`POST`	`/auth/refresh`	Renouvellement JWT via cookie
`POST`	`/auth/logout`	Révocation et déconnexion

04 — DONNÉES EXTERNES

Sources de données & pipeline de scraping

Règle architecturale actée : **le moins d'IA possible dans les pipelines de synchronisation**. Un sync complet porte sur 10 000+ cartes ; à 3-15s par appel LLM, un pipeline tout-IA prendrait 8 à 10h et monopoliserait la machine locale. L'IA est réservée aux fonctionnalités produit (scan, messagerie, rapports), jamais aux imports de masse.

Trois modes d'extraction, par ordre de préférence

Mode	Vitesse	Principe	Utilisé par
1 — CSS / JS évalué	~2-3s/carte	Playwright lit le DOM via sélecteurs CSS — déterministe, zéro IA	PriceCharting, 130point, CardLadder
2 — XHR intercept	Très rapide	Interception des appels réseau internes du site (JSON natif)	CGC Population (SPA, à venir)
3 — Ollama LLM	3-5× plus lent	Fallback si le HTML est trop complexe pour du CSS pur	PSA, CardMarket, Mavin

Providers en place

Provider	Mode	Données	Statut
PriceChartingProvider	CSS	RAW + PSA 1–10	Fait
PointSalesProvider (130point)	CSS	RAW + gradés multi-société	Fait
CardLadderProvider	CSS	Gradés PSA/BGS/CGC/SGC	Fait
TcgPlayerProvider	API REST	RAW USD	Fait
CardMarketScrapingProvider	Playwright + Ollama	RAW EUR	Fait
MavinProvider	Playwright + Ollama	RAW USD	Fait
PsaProvider	Playwright + Ollama	Population	Fait

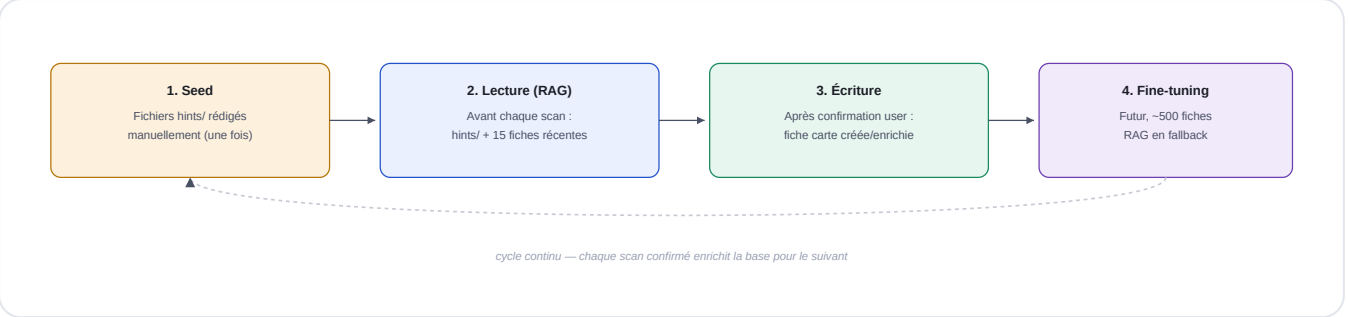
CONVENTION DE NOMMAGE

Trois syncs planifiés (Hangfire) sont utilisés partout dans le code, les logs et l'UI : **syncPrice** (`POST /api/v1/admin/sync/prices`), **syncTCG** (`/sync/catalog`) et **syncPOP** (`/sync/population`).

05 — ASSISTANT IA

AI-Brain — base de connaissance RAG

Le dossier `ai-brain/` (versionné dans le repo, déployé avec l'application) est la mémoire vivante des modules LLM produit — actuellement le scan de collection par photo. Elle est lue avant chaque appel LLM et enrichie après chaque confirmation utilisateur.



Positionnement de l'IA dans le produit

Contexte	IA ?	Phase
Pipelines de scraping (sync longue)	Non	MVP et au-delà
Scan photo de carte → identification	Oui — Vision LLM	Phase 4 — fait
Messagerie client → brouillon de réponse	Oui — LLM	Phase 2
Estimation de valeur de collection	Oui — à faire	Phase 4
Rapports automatiques boutique	Oui — à faire	Phase 4

Modèle utilisé : `llava:13b` (vision) via Ollama en local. Détail du prompt produit et des règles de reconnaissance de slab dans le document « Schéma de base de données » § CardInstance.

06 — REPRISE

Environnement dev & points d'attention

Ports & accès locaux (Docker)

Service	Port local	Détail
PostgreSQL	`5434`	Utilisateur `pokekool` / base `pokekool`
Redis	`6382`	Sans mot de passe (dev)
Adminer	`8091`	UI d'admin PostgreSQL
API .NET	`5080`	Swagger UI sur `/swagger`

Comptes seedés en dev : `admin@pokekool.dev` / `client@pokekool.dev` (voir `00-Index/Dev-Credentials.md` pour les mots de passe — fichier à ne jamais committer).

Dette technique & éléments hérités à trancher

SCAFFOLD MULTI-TENANT HÉRITÉ

Le README du repo et les endpoints Swagger exposent encore un login avec `tenantSlug` et des endpoints `/menus`, `/categories`, `/items` — vestiges d'un template générique multi-tenant (probablement un boilerplate de restauration, cf. l'ancien squelette mobile `Restau.Mobile` renommé en Pokekool.Mobile mi-2026). Ces éléments ne font pas partie du domaine Pokémon actuel. À trancher : nettoyer complètement, ou conserver le multi-tenant si une offre multi-boutiques est envisagée à terme (l'instance Pessac reste mono-tenant dans tous les cas).

DOCUMENTATION OBSIDIAN À METTRE À JOUR

`01-Vision/Stack-Technique.md` et le document fondateur `ARCHITECTURE.md` décrivent encore un frontend React — à corriger pour refléter le passage effectif à Angular 19 (voir §1). Les diagrammes de ce document utilisent déjà la stack réelle.

CE QUI EST SOLIDE ET RÉUTILISABLE EN L'ÉTAT

Auth JWT + Refresh (ADR-002), Clean Architecture (ADR-001), pipeline de scraping multi-mode, design system à source unique (`tokens.json`) partagé Web/Mobile, module Client et module Boutique Admin déjà fonctionnels de bout en bout.

Sources — coffre Obsidian « Projet_Pokemon » : 02-Architecture/*, 00-Index/Dev-Credentials.md ; repo `Pokekool` : README.md, ARCHITECTURE.md, package.json, src/. Document généré le 31 août 2026.